

Rapport — Jour 5

Intégration & Myhill–Nerode

Binôme : DOSSOU Persévérance & AGBOGBA Silas • Modules : `pipeline.py`, `formlang/myhill_nerode.py`

A. Réponses théoriques

Q5.1 (0,5) — Myhill–Nerode et lien avec le Jour 1

Pour un langage $L \subseteq \Sigma^*$, on pose $u \approx_L v$ ssi, pour tout suffixe $w \in \Sigma^*$, $uw \in L \Leftrightarrow vw \in L$. C'est une relation d'équivalence invariante à droite. Le théorème de Myhill–Nerode énonce que L est régulier ssi $\approx_L$ a un nombre fini de classes, et que ce nombre (l'indice de $\approx_L$) est exactement le nombre d'états de l'AFD minimal reconnaissant L .

Pour $L_1 = \{w \mid w \text{ contient le facteur } or\}$ (Jour 1), l'AFD minimal obtenu par la construction des sous-ensembles puis le raffinement de Moore possède $\boxed{3}$ états (A, B, C — cf. rapport Jour 1, Q1.2). Le calcul de `nerode_classes` sur des mots témoins retrouve exactement ces $\boxed{3}$ classes : le nombre de classes de $\approx_{L_1}$ et le nombre d'états de l'AFD minimal du Jour 1 coïncident, comme l'annonce le théorème.

Q5.2 (0,5) — suffixes témoins et fusion d'états

Deux mots u, v sont dits indistinguables par un ensemble de suffixes témoins S si, pour tout $s \in S$, $accepts(us) = accepts(vs)$ — c'est-à-dire que leurs signatures $(1[us \in L])_{s \in S}$ sont identiques. Ce critère est une *approximation calculable* de $\approx_L$: au lieu de tester tous les suffixes possibles (infinis), on se limite à un échantillon fini S de suffixes représentatifs (ici $\{\varepsilon, r, or\}$ pour L_1).

Le lien avec la minimisation (Jour 1, E1.2) est direct : à la minimisation, deux états de l'AFD complété sont fusionnés en un seul bloc dès qu'ils ne se distinguent jamais dans le futur, c'est-à-dire dès qu'aucun suffixe ne les sépare — exactement le critère « même signature sur les suffixes témoins ». Regrouper les mots par signature (`nerode_classes`) revient donc à retrouver, à partir des mots eux-mêmes, la même partition d'états que le raffinement de Moore obtient à partir de l'automate.

B. Exercices de programmation

E5.1

`pipeline.analyze_word` : chaîne leet_normalize (FST, Jour 1) $\rightarrow$ contains_or (AFD, Jour 1) $\rightarrow$ well_parenthesized (PDA, Jour 2) sur le mot normalisé. `pipeline.analyze_morpho` : discover (PRE/SUF sur le vocabulaire) $\rightarrow$ segment_to_tree $\rightarrow$ classify via le BUTA morphologique (Jour 3). Tests `test_pipeline_word`, `test_pipeline_morpho` : passés.

E5.3

`nerode_classes(accepts, words, suffixes)` : regroupe les mots par signature `tuple(accepts(w+s) for s in suffixes)` dans un dictionnaire, puis renvoie les classes (listes de mots de même signature). *Test `test_nerode_trois_classes` : passé.*

Sortie de référence obtenue

```
$ pytest -q
28 passed

$ python pipeline.py -word 4or
normalisé(FST) : aor
facteur_or(AFD) : True
délimiteurs_ok(PDA) : True
```

L'ensemble des 28 tests du projet (Jours 1 à 5) passe : le pipeline assemble les étages sans réécrire aucun automate, conformément à la règle gate.